Telemedicine Platform — Project Setup & Run Guide

From a fresh clone to a running application: prerequisites, configuration, commands, verification and troubleshooting

Covers: MySQL · FastAPI backend · React/Vite frontend · run.sh · AI keys (Groq, Gemini) · STUN/TURN
Default URLs: app http://localhost:5173 · API http://localhost:8000 · API docs http://localhost:8000/docs

Every command in this guide was run against a fresh copy of the repository (macOS, Python 3.11, Node 22, MySQL 9.7).
Shortest path: install the prerequisites, start MySQL, run **./run.sh**, set **DATABASE_URL** in backend/.env, run **./run.sh** again.

Contents

1. Prerequisites 3
2. Repository Setup 4
3. Environment Variables 5
4. Database Setup 7
5. Backend Setup 9
6. Frontend Setup 10
7. Running the Complete Project 11
8. AI API Configuration 13
9. WebRTC / STUN / TURN Setup 14
10. First Run Checklist 15
11. Common Setup Problems 16
12. Clean Restart / Reset 19
13. Production Reminder 20

1. Prerequisites

Verified on macOS (Apple silicon) with the versions in the last column. Linux should work the same way, since `run.sh` only uses standard tools, but it has not been tested there. On Windows, use WSL (Ubuntu), because `run.sh` is a bash script.

Requirement	Version needed	Check	Verified with
Python	3.10 or newer	`python3.11 --version`	3.11.15
Node.js	18, 20 or 22+ (Vite 6)	`node --version`	22.23
npm	comes with Node.js	`npm --version`	10.9
MySQL server	8.0 or newer	`mysql --version`	9.7.1
bash, shasum, sed	preinstalled on macOS/Linux	`bash --version`	macOS defaults
lsof (optional)	used for the port check	`which lsof`	macOS default
Browser	recent Chrome, Edge, Firefox or Safari	camera + microphone	Chrome

> On macOS the built-in **python3 is 3.9, which is too old**. Install a newer one (below). **run.sh** automatically picks python3.13, 3.12, 3.11 or 3.10 if any of them is installed.

Install on macOS (Homebrew)

```
brew install python@3.11 node mysql
brew services start mysql        # start MySQL now and at login
mysql_secure_installation        # optional: set a root password
```

Install on Ubuntu / WSL (not tested)

```
sudo apt update
sudo apt install python3.11 python3.11-venv mysql-server lsof
# Node.js 20+: install from nodejs.org or with nvm, then check: node --version
sudo systemctl start mysql
```

2. Repository Setup

Only the files that matter for running the project:

```
telemedicine/
|-- run.sh                  one command to install, prepare the database and start everything
|-- backend/
|   |-- requirements.txt    Python dependencies
|   |-- .env.example        template for backend/.env  (copied automatically by run.sh)
|   |-- .env                YOUR configuration - never commit (git-ignored)
|   |-- .venv/              Python virtual environment (created by run.sh)
|   |-- app/                FastAPI application  (entry point: app.main:app)
|   |   `-- database/init_db.py   creates database + tables + seed data
|   |-- uploads/            profile photos (created at runtime)
|   `-- recordings/         consultation audio awaiting transcription (runtime, private)
`-- frontend/
    |-- package.json        scripts: dev, build, preview
    |-- .env.example        template for frontend/.env  (copied automatically by run.sh)
    |-- vite.config.js      dev server on port 5173, proxies /api /ws /uploads -> backend
    `-- node_modules/       created by npm install
```

```
git clone <your-repository-url> telemedicine
cd telemedicine
chmod +x run.sh      # only if the execute permission was lost
```

3. Environment Variables

There are two files: **backend/.env** and **frontend/.env**. On the first run, `run.sh` copies each one from its .env.example. It also fills in a random **JWT_SECRET**, your **CLINIC_TIMEZONE** and the development passwords **ADMIN_PASSWORD=Admin@12345** and **DEMO_PASSWORD=Demo@12345**. The only value you must set yourself is **DATABASE_URL**.

Values exported in your shell take priority over backend/.env. Restart the backend after changing backend/.env, and restart the frontend after changing frontend/.env.

Backend — backend/.env

Variable	Required	Used for
DATABASE_URL	**Yes**	MySQL connection: `mysql+pymysql://USER:PASSWORD@HOST:PORT/DATABASE`
JWT_SECRET	**Yes** (auto)	Signs login tokens. At least 32 characters; run.sh generates one.
JWT_EXPIRE_MINUTES	No (480)	How long a login lasts.
CORS_ORIGINS	No	Allowed browser origins, only needed if the frontend does not use the Vite proxy.
CLINIC_TIMEZONE	No (auto)	IANA time zone in which doctors' weekly hours are entered, e.g. Asia/Kolkata.
BOOKING_HORIZON_DAYS	No (60)	How far ahead patients can book.
CONSULTATION_JOIN_WINDOW_MINUTES	No (15)	How early before / late after an appointment the video room opens.
STUN_SERVER	No (Google STUN)	STUN URL(s) for video calls, comma-separated.
TURN_SERVER / TURN_USERNAME / TURN_PASSWORD	No	TURN relay for restrictive networks (section 9).
ICE_TRANSPORT_POLICY	No (all)	`all`, or `relay` to force TURN while testing.
ADMIN_EMAIL / ADMIN_PASSWORD / ADMIN_NAME	For an admin login	Admin account created on first database setup.
SEED_DEMO_DATA / DEMO_PASSWORD	For demo logins	Create demo doctors/patients; DEMO_PASSWORD is their shared password.
MAX_UPLOAD_MB	No (2)	Profile photo size limit.
GROQ_API_KEY	For AI notes	Speech-to-text. Empty = recording/AI feature hidden.
GROQ_WHISPER_MODEL	No	Default `whisper-large-v3`.
GEMINI_API_KEY	For AI notes	Structured summary generation.
GEMINI_MODEL	No	Default `gemini-2.5-flash`.
MAX_RECORDING_MB	No (25)	Per-file limit for uploaded consultation audio.

Not in .env.example but read if set: `JWT_ALGORITHM` (default HS256), `APP_NAME`, `UPLOAD_DIR`, `RECORDINGS_DIR`. You normally don't need them.

Frontend — frontend/.env

Variable	Required	Used for
VITE_API_BASE_URL	No (/api)	REST base URL. Keep `/api` to use the Vite dev proxy.
VITE_PROXY_TARGET	No (http://localhost:8000)	Where the dev server forwards /api, /ws and /uploads. Change it if the backend runs on another port.
VITE_CURRENCY	No (USD)	Currency code used to display consultation fees.

Shell variables used by run.sh and tests

Variable	Used for
API_PORT	Backend port for `./run.sh` / `./run.sh backend` (default 8000).
WEB_PORT	Frontend port for `./run.sh` / `./run.sh frontend` (default 5173).
TEST_DATABASE_URL	Database for integration tests. **It is dropped and recreated**, so use a separate database.

> Never commit .env files or real keys. Both .env files are already in .gitignore. API keys belong only in backend/.env; the frontend never needs them.

4. Database Setup

The project has **no SQL scripts and no migration tool**. The database, all tables and seed data are created by one Python module, `app.database.init_db`, which `./run.sh` and `./run.sh db` run for you. It is safe to run repeatedly.

Step 1 — Start MySQL

```
# macOS
brew services start mysql          # or: mysql.server start
# Linux
sudo systemctl start mysql
# check it is running
mysqladmin -u root -p ping         # expected: mysqld is alive
```

Step 2 — Choose credentials (pick one option)

Option A – use root (simplest for local development). If root has a password, use it in DATABASE_URL. The database is created automatically.

Option B – dedicated user (recommended on Ubuntu, where root often only works through sudo):

```
sudo mysql -u root -p
CREATE DATABASE telemedicine CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'telemed'@'localhost' IDENTIFIED BY 'choose-a-password';
GRANT ALL PRIVILEGES ON telemedicine.* TO 'telemed'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Step 3 — Set DATABASE_URL in backend/.env

```
# Option A
DATABASE_URL=mysql+pymysql://root:YOUR_ROOT_PASSWORD@127.0.0.1:3306/telemedicine
# Option A, root without a password
DATABASE_URL=mysql+pymysql://root@127.0.0.1:3306/telemedicine
# Option B
DATABASE_URL=mysql+pymysql://telemed:choose-a-password@127.0.0.1:3306/telemedicine
```

If the password contains special characters such as @, : or /, URL-encode them (for example @ becomes %40).

Step 4 — Create tables and seed data

```
./run.sh db
```

Expected output on the first run:

```
==> Preparing database
Preparing database…
  + 15 specializations
  + admin account admin@example.com
  + demo data: 12 doctors, 3 patients
Database ready.
```

On later runs only 'Preparing database…' and 'Database ready.' appear, because existing data is kept. Tables are created from the application's models; existing tables are never altered.

Seed data and logins created

Account	Email	Password
Admin	admin@example.com (ADMIN_EMAIL)	ADMIN_PASSWORD (run.sh default: Admin@12345)
Patient	patient@example.com	DEMO_PASSWORD (run.sh default: Demo@12345)
Doctor	sarah.mitchell@example.com	DEMO_PASSWORD

Demo data is only created when SEED_DEMO_DATA=true and DEMO_PASSWORD is set **at the time of the first seed**. Changing the passwords in .env later does not change existing accounts.

Step 5 — Verify the database

```
mysql -u root -p telemedicine -e "SHOW TABLES;"
# expected 10 tables: appointments, consultation_documentation, consultations,
# doctor_availability, doctor_profiles, doctor_time_off, notifications,
# patient_profiles, specializations, users

curl http://localhost:8000/api/health     # once the backend runs: {"status":"ok"}
```

5. Backend Setup

`./run.sh` performs all of these steps automatically. Use the manual steps if you want to run the backend on its own or understand what happens.

Automatic

```
./run.sh backend      # venv + dependencies + .env if missing, then starts the API
```

`./run.sh backend` does not create tables. Run `./run.sh db` (or the full `./run.sh`) once first.

Manual

```
cd backend
python3.11 -m venv .venv
source .venv/bin/activate              # Windows/WSL: same command inside WSL
pip install --upgrade pip
pip install -r requirements.txt

cp .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"   # paste as JWT_SECRET
# edit .env: DATABASE_URL, JWT_SECRET, ADMIN_PASSWORD, DEMO_PASSWORD

python -m app.database.init_db          # database + tables + seed data
uvicorn app.main:app --reload --port 8000
```

Run these commands from inside the `backend` folder, because `app.main` is imported from there.

What	URL
API base	http://localhost:8000/api
Health check	http://localhost:8000/api/health -> {"status":"ok"}
Interactive API docs	http://localhost:8000/docs

The backend listens on 127.0.0.1 only, so it isn't reachable from other devices by default.

6. Frontend Setup

Automatic

```
./run.sh frontend     # npm install + .env if missing, then starts Vite
```

Manual

```
cd frontend
npm install
cp .env.example .env        # optional: the defaults are the same values
npm run dev                 # the 'dev' script in package.json (vite)
```

Script (package.json)	Command	Purpose
dev	`npm run dev`	Development server on **http://localhost:5173** with the API proxy. Use this.
build	`npm run build`	Production build into frontend/dist (used by ./run.sh test as a check).
preview	`npm run preview`	Serves dist/ for a quick look. No API proxy is configured for it, so API calls will not work.

The backend must be running for the app to work. In development the browser only talks to localhost:5173, and Vite forwards API, WebSocket and upload requests to the backend.

7. Running the Complete Project

```
./run.sh
```

Prerequisite: MySQL must already be running and DATABASE_URL must be set. On the very first run, `run.sh` creates backend/.env and stops with a message asking you to set DATABASE_URL; set it and run `./run.sh` again.

What it does, in order

1. Creates `backend/.venv` with Python 3.10+ (if missing) and installs `requirements.txt` (only when it changed).
2. Creates `backend/.env` from `.env.example` (if missing) with a generated JWT_SECRET.
3. Runs `npm install` in `frontend/` (if needed) and creates `frontend/.env` (if missing).
4. Stops with a clear error if DATABASE_URL is still the placeholder.
5. Creates the database (if allowed), tables and seed data.
6. Checks that ports 8000 and 5173 are free; if not, shows the process using them.
7. Starts the FastAPI backend (auto-reload) and the Vite frontend together.

Expected output

```
==> Preparing database
Preparing database…
Database ready.
==> API:     http://localhost:8000  (interactive docs: /docs)
==> Web app: http://localhost:5173   - press Ctrl+C to stop both
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
  VITE v6.4.4  ready in ... ms
  ->  Local:   http://localhost:5173/
INFO:     Application startup complete.
```

	URL
Open the app	**http://localhost:5173**
Backend / API docs	http://localhost:8000 / http://localhost:8000/docs
Stop everything	Press **Ctrl+C** in the terminal; both servers stop.

All run.sh commands

Command	What it does
`./run.sh`	Install what is missing, prepare the database, start backend + frontend.
`./run.sh setup`	Install dependencies and create both .env files only.
`./run.sh db`	Create database, tables and seed data (safe to repeat).
`./run.sh demo-call`	Create a confirmed appointment starting now, to test a video call immediately.
`./run.sh backend`	Start only the backend.

Command	What it does
`./run.sh frontend`	Start only the frontend.
`./run.sh test`	Backend tests and a frontend production build. Integration tests run only if TEST_DATABASE_URL is set (otherwise 7 pass, 15 are skipped).
`API_PORT=8001 WEB_PORT=5174 ./run.sh`	Use other ports. Also set VITE_PROXY_TARGET=http://localhost:8001 in frontend/.env.

Quick functional check

1. Open http://localhost:5173 and sign in as **patient@example.com** / DEMO_PASSWORD. The patient dashboard opens.
2. Find Doctors shows 10 verified doctors, so the backend API connection works.
3. Sign in as **admin@example.com** / ADMIN_PASSWORD in another tab. The admin dashboard shows 2 doctors awaiting verification.

8. AI API Configuration

Optional. Without keys, everything works except AI-assisted notes; the doctor's **Record** button is simply hidden and manual notes remain available.

```
# backend/.env
GROQ_API_KEY=<your Groq key>        # https://console.groq.com/keys    -> speech-to-text
GEMINI_API_KEY=<your Gemini key>    # https://aistudio.google.com/apikey -> structured notes
GROQ_WHISPER_MODEL=whisper-large-v3
GEMINI_MODEL=gemini-2.5-flash
```

Restart the backend after adding keys (stop with Ctrl+C, then `./run.sh`). The keys are read only by the backend; they are never sent to the browser.

Key	Used by	If missing
GROQ_API_KEY	Transcribing the recorded consultation audio after the call ends	Record button hidden; upload rejected
GEMINI_API_KEY	Turning the transcript into the structured draft for the doctor	Transcript is produced, the summary step fails; the doctor can write it manually

Verify the keys

```
# 1. Keys are valid (expected HTTP 200 and a JSON list of models)
curl -s -o /dev/null -w "%{http_code}\n" https://api.groq.com/openai/v1/models \
     -H "Authorization: Bearer $GROQ_API_KEY"
curl -s -o /dev/null -w "%{http_code}\n" https://generativelanguage.googleapis.com/v1beta/models \
     -H "x-goog-api-key: $GEMINI_API_KEY"
#    invalid Groq key   -> 401  "Invalid API Key"
#    invalid Gemini key -> 400  "API key not valid"

# 2. The backend sees them (needs a login token)
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login -H "Content-Type: application/json" \
  -d '{"email":"sarah.mitchell@example.com","password":"Demo@12345"}' | python3 -c "import sys,json;print(jso
curl -s http://localhost:8000/api/documentation/config -H "Authorization: Bearer $TOKEN"
#    expected: {"transcription_enabled":true,"summarization_enabled":true}
```

In step 1, `$GROQ_API_KEY` and `$GEMINI_API_KEY` mean your keys: export them in the terminal first or paste them in place. **End-to-end check:** start a call (section 9), press **Record** as the doctor, speak, end the call. The doctor's screen shows Upload → Transcribe → Draft summary, then the editable draft and the transcript.

9. WebRTC / STUN / TURN Setup

For local testing **no configuration is needed**. The default STUN server is already set, and two browser windows on the same computer connect directly.

Variable (backend/.env)	Local development value	When to change
STUN_SERVER	stun:stun.l.google.com:19302 (default)	Rarely; comma-separate several URLs if wanted.
TURN_SERVER	empty	When users are on different, restrictive networks (corporate/hospital Wi-Fi, some mobile networks).
TURN_USERNAME / TURN_PASSWORD	empty	Credentials of your TURN server.
ICE_TRANSPORT_POLICY	all	Set to `relay` only to test that TURN works; it is ignored if no TURN server is set.

```
# example TURN configuration (values from your TURN provider)
TURN_SERVER=turn:turn.example.com:3478?transport=udp,turns:turn.example.com:5349
TURN_USERNAME=<username>
TURN_PASSWORD=<password>
```

Test a video call locally

1. Run `./run.sh demo-call` in a second terminal. It prints the appointment it created.
2. Open http://localhost:5173 in two tabs (or a normal and a private window). Sign in as patient@example.com in one and sarah.mitchell@example.com in the other. Each tab keeps its own login.
3. In both: open the appointment (dashboard → Join consultation) → allow camera and microphone → **Join consultation**.
4. The top bar shows **Connected · Direct connection**. With ICE_TRANSPORT_POLICY=relay and TURN set, it shows **Relayed via TURN**.

Local-development considerations

- **Camera/microphone need a secure context.** http://localhost works. Opening the app from another device by IP address over plain http will not get camera access; that requires HTTPS.
- Both servers listen on localhost only, so other devices on your network can't reach them without extra configuration.
- **Permissions:** allow camera and microphone when the browser asks. If you blocked them, re-enable them from the camera icon / site settings in the address bar and click Try again.
- Chrome lets two tabs share one camera. Some browsers do not; if the second tab shows 'camera in use', use two different browsers or 'Join without camera and microphone'.
- The Join button appears 15 minutes before an appointment starts (CONSULTATION_JOIN_WINDOW_MINUTES). Use `./run.sh demo-call` to get one that is joinable immediately.

10. First Run Checklist

- ■ Python 3.10+, Node.js 18+/20+/22+, MySQL 8+ installed (section 1)
- ■ MySQL running: `mysqladmin -u root -p ping` → mysqld is alive
- ■ DATABASE_URL set in backend/.env
- ■ `./run.sh` completed: dependencies installed, both .env files created, 'Database ready.' printed
- ■ Backend starts: http://localhost:8000/api/health returns {"status":"ok"}
- ■ Frontend starts: http://localhost:5173 opens the sign-in page
- ■ Login works with patient@example.com / DEMO_PASSWORD
- ■ API connection works: Find Doctors lists the demo doctors
- ■ Real-time works: as the doctor, accept a request; the patient tab shows a notification without reloading
- ■ Video call works: `./run.sh demo-call`, two tabs, both show 'Connected'
- ■ (Optional) STUN/TURN configured for networks outside your computer
- ■ (Optional) GROQ_API_KEY and GEMINI_API_KEY set; /api/documentation/config returns true for both
- ■ (Optional) AI transcription works: record a call, the transcript appears
- ■ (Optional) AI documentation works: the draft appears, can be edited and approved, and the patient sees the visit summary

11. Common Setup Problems

Problem	`Error: Python 3.10 or newer is required.`
Likely cause	Only an older Python (e.g. macOS's built-in 3.9) is installed.
Fix	`brew install python@3.11` (macOS) or `sudo apt install python3.11 python3.11-venv`, then run `./run.sh` again.

Problem	`Error: Node.js 18+ and npm are required.` or `npm: command not found`
Likely cause	Node.js is not installed or not on PATH.
Fix	`brew install node` or install Node 20+ from nodejs.org / nvm. Check with `node --version`.

Problem	Dependency installation fails (pip or npm)
Likely cause	No internet connection or proxy, an outdated Node version, or a broken partial install.
Fix	Check your connection, then reinstall cleanly: `rm -rf backend/.venv frontend/node_modules && ./run.sh setup`.

Problem	`Error: Set DATABASE_URL in backend/.env ...`
Likely cause	backend/.env still contains the placeholder value from .env.example.
Fix	Edit DATABASE_URL in backend/.env (section 4, step 3) and run `./run.sh` again.

Problem	`(2003, "Can't connect to MySQL server ... Connection refused")`
Likely cause	MySQL is not running, or the host/port in DATABASE_URL is wrong.
Fix	Start MySQL (`brew services start mysql` / `sudo systemctl start mysql`), check the port (default 3306).

Problem	`(1045, "Access denied for user ...")`
Likely cause	Wrong user or password in DATABASE_URL. On Ubuntu, root often only works through sudo.
Fix	Fix the credentials, or create a dedicated user (section 4, option B). URL-encode special characters in the password.

Problem	`Could not create database automatically` followed by `Unknown database`
Likely cause	The MySQL user is not allowed to create databases.
Fix	Create it once as root: `CREATE DATABASE telemedicine CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;` and grant privileges.

Problem	`Error: Port 8000 (API) is already in use by PID ...`
Likely cause	Another program, or an earlier run that was not stopped, is using the port.
Fix	Run the `kill <PID>` shown in the message, or use other ports: `API_PORT=8001 WEB_PORT=5174 ./run.sh` and set VITE_PROXY_TARGET=http://localhost:8001 in frontend/.env.

Problem	Backend fails at startup: `JWT_SECRET must be at least 32 characters long` or `Field required`
Likely cause	JWT_SECRET or DATABASE_URL is missing or too short in backend/.env.
Fix	Generate a secret: `python3 -c "import secrets; print(secrets.token_urlsafe(48))"`, paste it as JWT_SECRET, restart.

Problem	App shows 'Unable to reach the server' or every page shows an error
Likely cause	The backend is not running, or VITE_PROXY_TARGET points to the wrong port.
Fix	Check http://localhost:8000/api/health. Make sure VITE_PROXY_TARGET matches the backend port, then restart the frontend.

Problem	CORS error in the browser console
Likely cause	VITE_API_BASE_URL was set to an absolute URL (e.g. http://localhost:8000/api), so the proxy is bypassed.
Fix	Use the default `VITE_API_BASE_URL=/api`, or add the frontend's origin to CORS_ORIGINS in backend/.env and restart.

Problem	Notifications don't arrive live, or the video room stays on 'Joining…' / 'Reconnecting…'
Likely cause	The WebSocket can't connect: backend down, or the page was opened on a different port than the proxy.
Fix	Open the app through http://localhost:5173 (not :8000), confirm the backend is running, reload the page.

Problem	Demo or admin login fails with 'Invalid email or password'
Likely cause	DEMO_PASSWORD/ADMIN_PASSWORD were empty when the database was first seeded, or were changed afterwards.
Fix	Sign in with the password that was set at the first seed, or reset the local database (section 12) and run `./run.sh db`.

Problem	'Camera and microphone access was blocked' or 'Video calls need a secure connection'
Likely cause	Permission was denied, or the app was opened over plain http from another device.
Fix	Allow camera/microphone in the address bar's site settings and click Try again. Use http://localhost on the same computer.

Problem	'Your camera or microphone is being used by another application'
Likely cause	Another app or browser holds the camera.
Fix	Close the other app, or join the second participant from a different browser.

Problem	No 'Join consultation' button
Likely cause	The appointment is not confirmed yet, or it starts more than 15 minutes from now.
Fix	Accept it as the doctor, wait until the join window, or use `./run.sh demo-call`.

Problem	Call shows 'Unable to connect' (STUN/TURN failure)
Likely cause	The two users are on networks that block direct connections, and no TURN server is configured, or the TURN credentials are wrong.
Fix	Configure TURN_SERVER/USERNAME/PASSWORD (section 9). Test with ICE_TRANSPORT_POLICY=relay; the status must show 'Relayed via TURN'.

Problem	Doctor doesn't see the Record button
Likely cause	GROQ_API_KEY is empty, the backend wasn't restarted after adding it, or you're signed in as a patient.
Fix	Set the key, restart the backend, check /api/documentation/config (section 8).

Problem	Documentation shows 'Speech-to-text failed (401): Invalid API Key'
Likely cause	GROQ_API_KEY is wrong or revoked.
Fix	Replace it, restart the backend, then press Retry in the documentation panel.

Problem	Documentation shows 'Summarization failed (400/403/429) ...'
Likely cause	GEMINI_API_KEY invalid, Gemini not enabled for the key, or quota exceeded (429).
Fix	Check the key with the curl in section 8, wait if it is a quota error, restart the backend and press Retry. The transcript is kept; the doctor can also write the summary manually.

12. Clean Restart / Reset

Goal	Command
Stop everything	Ctrl+C in the run.sh terminal
Stop a leftover server	`lsof -ti :8000 | xargs kill` (or :5173)
Restart backend only	`./run.sh backend`
Restart frontend only	`./run.sh frontend`
Recreate the Python environment	`rm -rf backend/.venv && ./run.sh setup`
Reinstall frontend dependencies	`rm -rf frontend/node_modules && ./run.sh setup`
Recreate backend/.env from the template	`mv backend/.env backend/.env.bak && ./run.sh setup`, then set DATABASE_URL again. A new JWT_SECRET signs everyone out.
Re-run database setup (safe)	`./run.sh db` (creates anything missing, keeps existing data)
Fresh demo appointment for a call	`./run.sh demo-call`

DESTRUCTIVE – deletes all local data (users, appointments, consultations, notes, uploaded photos). Only for a local development database you no longer need. It recreates the database with fresh seed data; use your own database name if it differs.

```
mysql -u root -p -e "DROP DATABASE telemedicine;"
rm -rf backend/uploads backend/recordings
./run.sh db
```

13. Production Reminder

- Never commit .env files; set production values through your hosting environment.
- Never put API keys (Groq, Gemini, TURN) in frontend code; they belong only in the backend environment.
- Use a dedicated MySQL user with a strong password, not root, and take backups.
- Generate a new, long JWT_SECRET and replace the development ADMIN_PASSWORD; set SEED_DEMO_DATA=false.
- Serve everything over HTTPS (camera access and secure WebSockets require it).
- Configure your own TURN server (with TLS on port 443) in addition to STUN.
- Build the frontend with `npm run build` and serve it with a web server that also forwards /api and /ws to the backend.